Modèles algorithmiques fondamentaux en C++

Thème principal : Initiation au développement et structures algorithmiques de base

Synthèse exécutive : Ce cours présente les trois modèles algorithmiques incontournables à maîtriser en BUT 1 Informatique pour réussir les SAE : le parcours complet avec traitement systématique, le parcours complet avec traitement conditionné, et la recherche de première occurrence. L'accent est mis sur l'analyse rigoureuse préalable de la structure de données (principalement les structures à accès direct comme les tableaux et chaînes de caractères), la gestion correcte des bornes d'indices, et le choix de la variante adaptée selon que l'on cherche une simple existence ou une position exacte.

1. Le Pourquoi et les Mécanismes d'Accès aux Structures de Données

Avant de concevoir le moindre algorithme, il est impératif d'identifier la nature de la collection de données manipulée. Le choix du modèle algorithmique dépend directement des opérations permises par la structure. Les modèles présentés dans ce cours s'appliquent principalement aux structures à accès direct (tableaux, chaînes de caractères), où l'on peut cibler n'importe quel indice immédiatement. Appliquer un modèle à accès direct sur un flux purement séquentiel (comme un fichier CSV brut ou un fichier texte) invalide totalement l'algorithme.

- Structure de données : Collection regroupant des éléments de même type (ex: tableaux, chaînes de caractères).
- Accès direct : Mécanisme permettant d'atteindre instantanément n'importe quel élément via son indice ou sa clé, sans parcourir les précédents.
- Accès séquentiel : Mécanisme imposant de se positionner au début puis de progresser pas à pas d'un élément au suivant.
- Accès dynamique : Structure proposant à la fois des mécanismes d'accès séquentiel et d'accès direct.

Cas concret enseignant : *Le professeur insiste sur une erreur classique commise en partiel : appliquer un modèle nécessitant un accès direct (par exemple, un accès par indice i dans un tableau) sur un fichier texte ou un flux séquentiel, ce qui provoque une erreur de conception majeure.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle est le plus simple mais exige une rigueur absolue sur la gestion des indices. Il est utilisé lorsque l'on doit inspecter et modifier ou afficher l'intégralité d'une zone de données (par exemple, remplir toutes les cases d'un tableau ou mettre en majuscules tous les caractères d'une chaîne). Le piège classique des étudiants est de faire varier la boucle jusqu'à N au lieu de N - 1, ce qui provoque une erreur fatale de dépassement de capacité (out of bounds).

- Objectif : Appliquer un traitement à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : Boucle bornée (pour / for).
- Gestion des bornes : Sur un tableau de N éléments en notation standard, les indices vont de 0 à N - 1.

Cas concret enseignant : *Exemple concret donné en amphi : Afficher un tableau de N cases. L'indice 'i' varie de 0 à N - 1. Le traitement répété consiste simplement à afficher l'élément du tableau situé à la position 'i'.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionnel

C'est une évolution directe du modèle précédent. Le parcours reste complet (on visite tous les éléments de la borne A à la borne B), mais l'action à l'intérieur de la boucle est conditionnée par un test logique. Par exemple, compter le nombre de voyelles dans une chaîne de caractères ou afficher uniquement les nombres positifs d'un tableau.

- Objectif : Parcourir l'ensemble d'une structure (de la borne de début à la fin) mais n'appliquer le traitement qu'aux éléments vérifiant une propriété spécifique.

- Structure : Boucle bornée (pour) englobant une structure alternative (si / if).
- Complexité des conditions : La condition peut être simple ou complexe.

Cas concret enseignant : *Analogie du contrôle qualité : Sur une chaîne de montage (le parcours complet), on examine chaque pièce, mais on ne retire de la chaîne (traitement) que les pièces présentant un défaut précis.*

4. Troisième Modèle : La Recherche de Première Occurrence

Contrairement aux deux premiers modèles, il ne s'agit pas de parcourir forcément toute la structure, mais de s'arrêter dès que l'objectif est atteint. C'est un algorithme pessimiste par nature : on initialise un booléen 'trouvé' à faux, considérant que l'élément n'est pas là, et on ajuste notre certitude au fil de la visite. Rédiger un algorithme de recherche avec une seule condition d'arrêt est une erreur fondamentale en évaluation.

- Objectif : Trouver le tout premier élément d'une structure qui vérifie une propriété donnée.
- Itération non bornée : Utilisation d'une boucle conditionnelle (tant que / while).
- Double condition d'arrêt obligatoire : (1) A-t-on épuisé tous les éléments ? (2) A-t-on trouvé l'élément recherché ?
- Variante 1 (Rupture immédiate) : Idéale pour récupérer la position exacte (indice).
- Variante 2 (Test unique en tête) : Plus simple à structurer syntaxiquement avec un 'tant que' global.

Cas concret enseignant : *Le professeur rappelle qu'en informatique, il faut être pessimiste : on part du principe que l'élément n'est pas dans la structure, et on modifie notre indicateur booléen dès qu'on le rencontre.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, à condition de respecter les mécanismes d'itération propres à cette structure, mais il faut d'abord convertir ou adapter le parcours selon que l'on manipule la partie séquentielle ou directe de la structure dynamique.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La première variante avec rupture immédiate permet d'obtenir directement l'indice exact de l'élément, tandis que la seconde variante avec un 'tant que' global simplifie l'écriture logique si l'on cherche uniquement un booléen d'existence.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut adapter les bornes de la boucle pour éviter les dépassements (par exemple commencer à 1 et s'arrêter à N-2) afin de pouvoir comparer T[i] avec T[i-1] ou T[i+1] en toute sécurité.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en utilisant le modèle approprié.

Résolution pas à pas :
1. Analyse du problème : On doit traiter tous les éléments d'un tableau (accès direct) de la première à la dernière case, mais le traitement (l'affichage) ne doit s'exécuter que si la valeur est strictement positive. Le modèle adapté est donc le 'Parcours complet avec traitement conditionnel'. 2. Détermination des bornes : Le tableau possède 'N' cases, indicées de 0 à N - 1. La borne de début est 0, la borne de fin est N - 1. 3. Construction de la boucle : Une boucle bornée 'pour i allant de 0 à N - 1'. 4. Ajout de la condition : À l'intérieur de la boucle, on ajoute un test : 'si T[i] > 0 alors'. 5. Action : 'Afficher(T[i])'.

Résultat : Pour i allant de 0 à N - 1 faire Si T[i] > 0 alors Afficher(T[i]) Fin Si Fin Pour Ce code respecte strictement les prérequis d'un parcours complet conditionné sur structure à accès direct.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on utiliser pour vérifier si un tableau contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence (ou recherche d'existence avec arrêt anticipé).

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce que les indices des tableaux en C++ commencent à 0, ce qui signifie que le nième et dernier élément se trouve à l'indice N - 1.

Q : Combien de conditions d'arrêt minimales doit comporter un algorithme de recherche de première occurrence ?

R : Deux conditions d'arrêt : l'épuisement de la structure (fin du parcours) et la trouvaille de l'élément recherché.